

NPTEL Course on "Computer System Optimization for ML"

Lecture #20

NPTEL

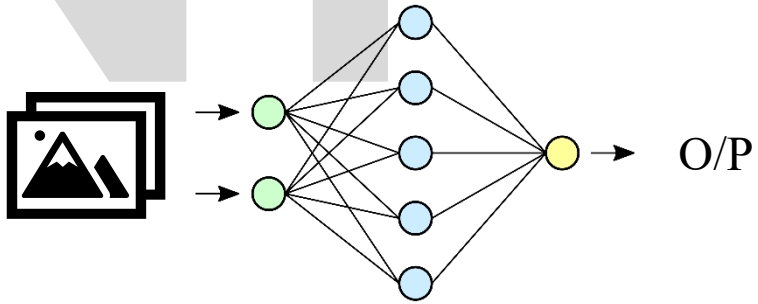
Agenda

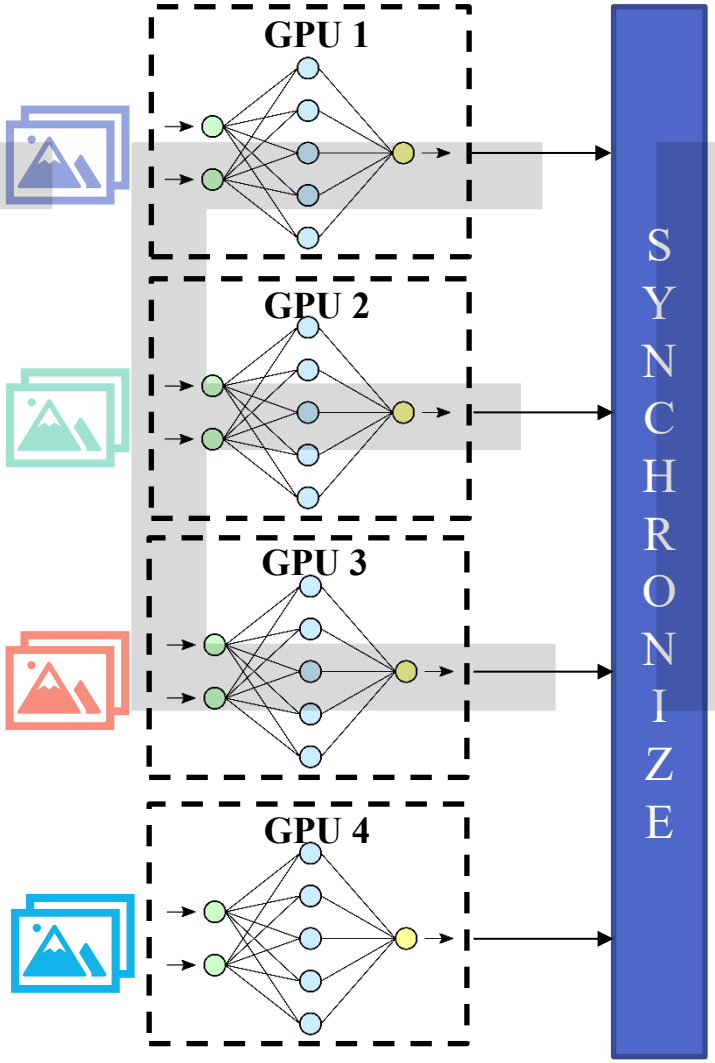
- Recap
- Parallelism in DNN Training (contd.)

NPTEL

Data Parallel Training

- Replicate the model across all GPUs
- Shard (break) the inputs
- Forward and backward propagation on same model but with different data
- Average parameters (weights/bias), updates (gradients) at every iteration

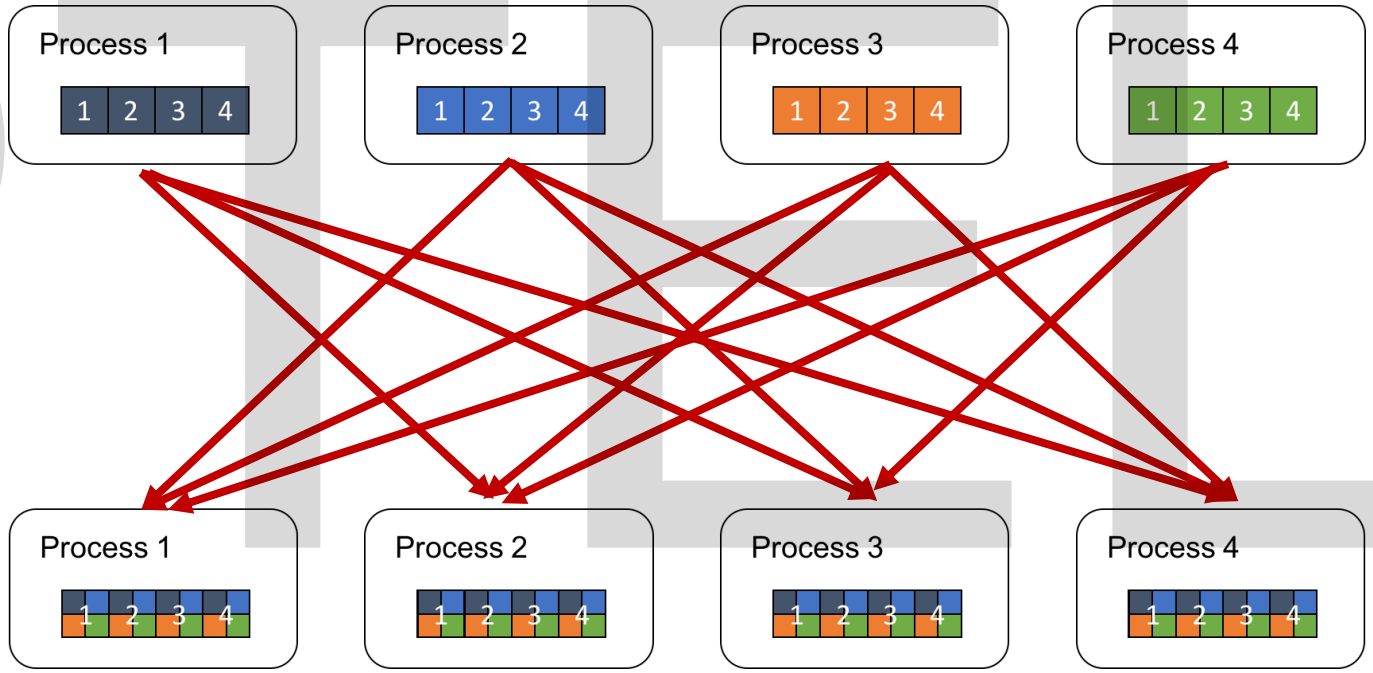




NPTEL

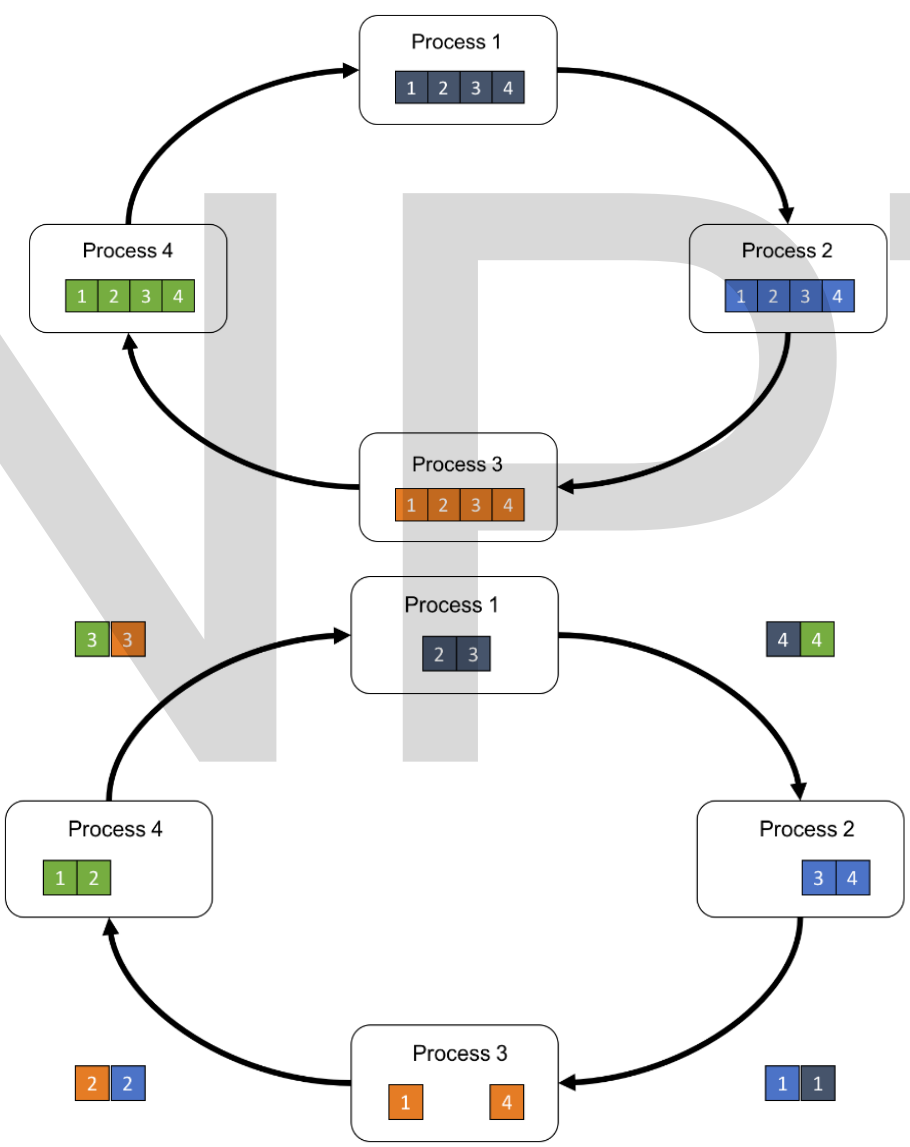
Reducing Synchronization Overhead

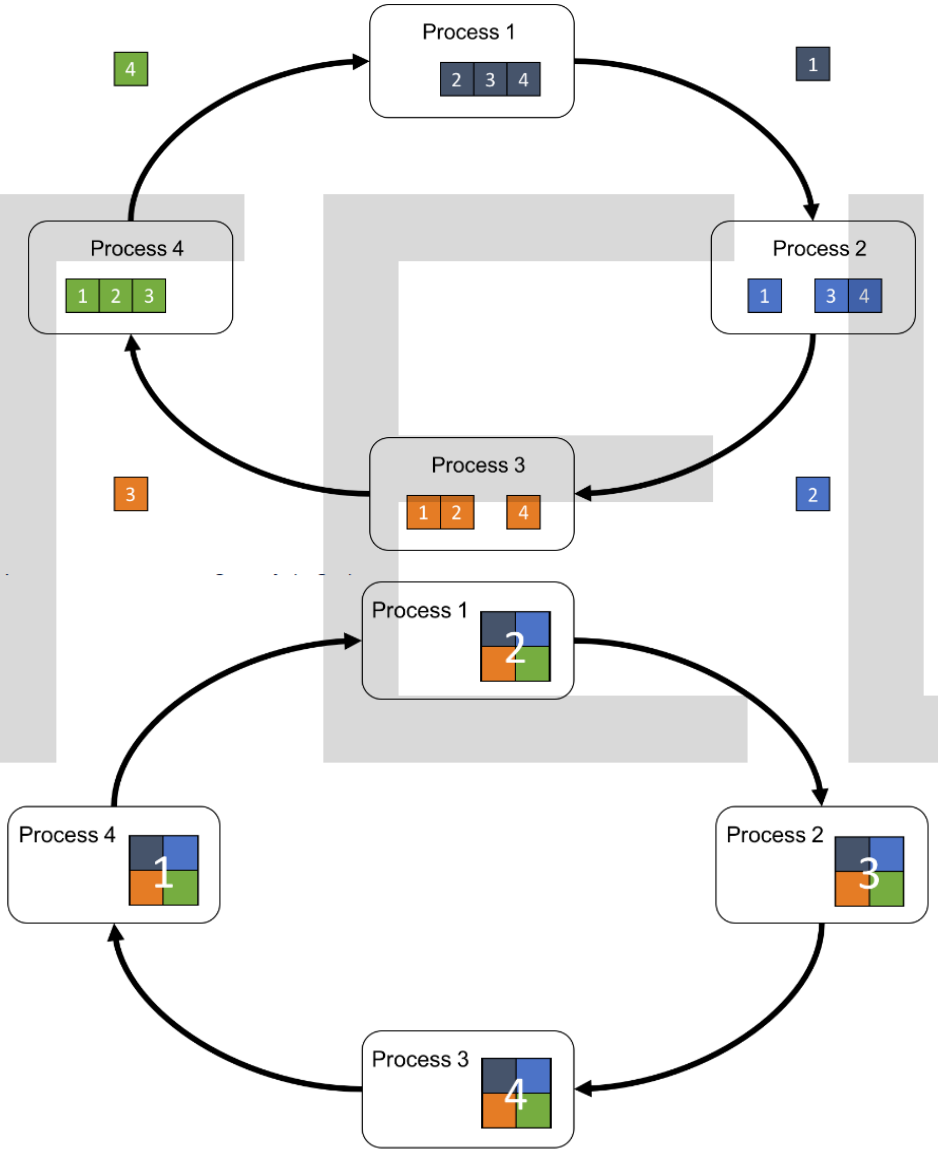
- **NCCL All Reduce**
  - NVIDIA Collective Communication Library (NCCL)
- **Simple strategy**
  - All-to-all communication



What is the amount of the all to all communication if there are P processes?

Ring-AllReduce





Agenda

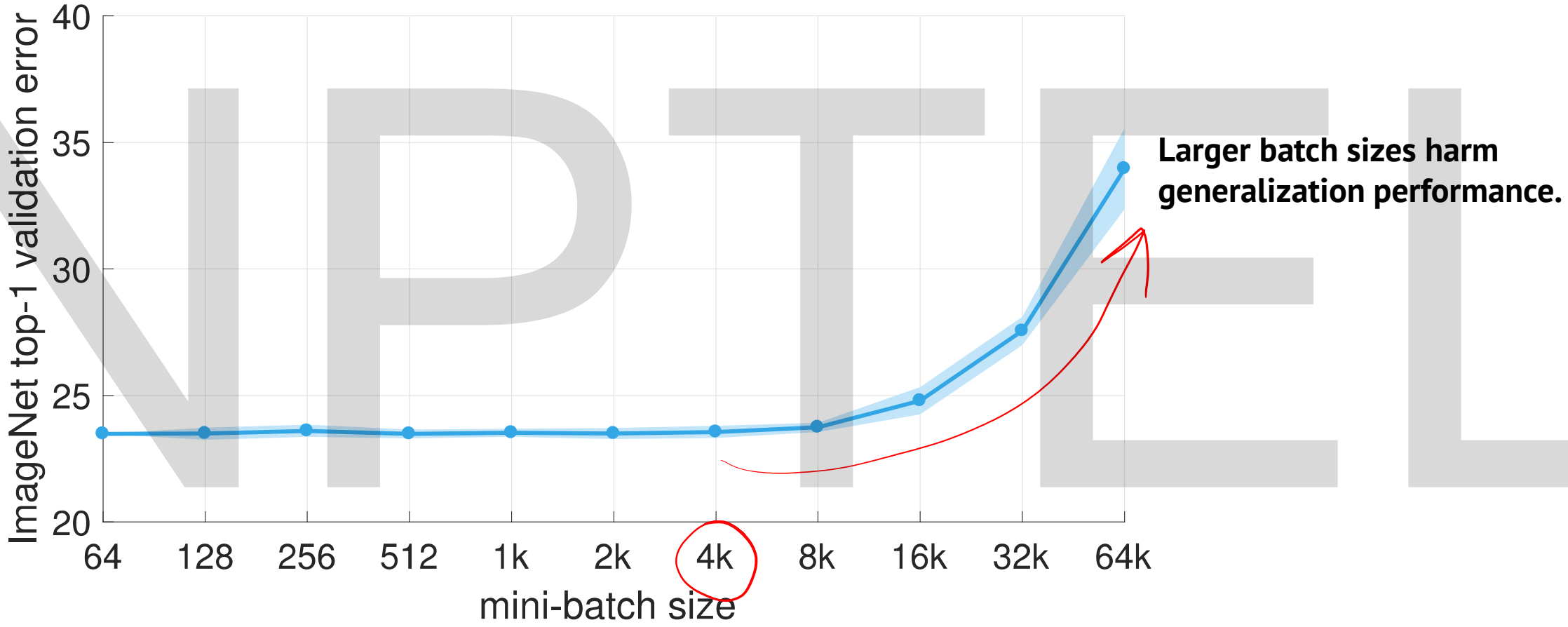
- Recap
- **Parallelism in DNN Training (contd.)**

NPTEL

If we want to keep scaling synchronous SGD then we have to keep increasing the batch size.

NPTEL

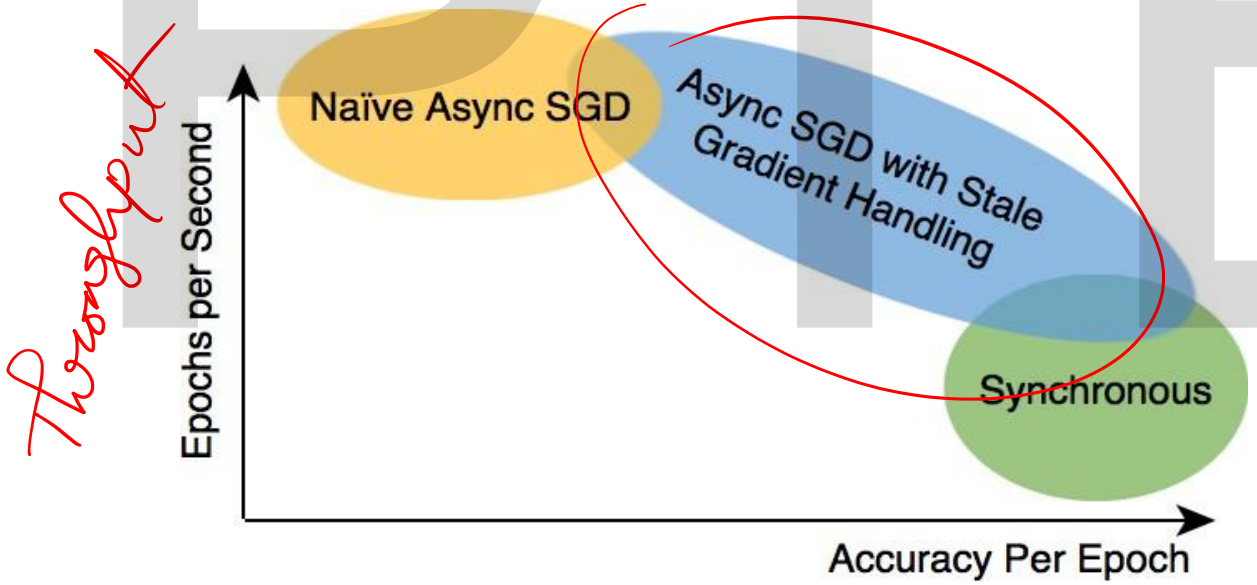
Generalization Gap Problem



Goyal, Priya, et al. "Accurate, large minibatch SGD: Training imagenet in 1 hour." arXiv preprint arXiv:1706.02677 (2017).

No Free Lunch

- Perfect linear scaling between performance with workers is impossible
- Common problems: communication overhead (synchronous), stalled gradients (asynchronous)



NPTEL

Data Parallelism Summary

- **An efficient training method where the communication time is independent of GPU workers with ring AllReduce**
- **Super easy to implement - add an allReduce before parameter update**
- **Challenging to scale**
  - Balancing statistical and hardware efficiency is difficult
- **Does not work for large models**
- **Communication overhead could be prominent**

NPTEL

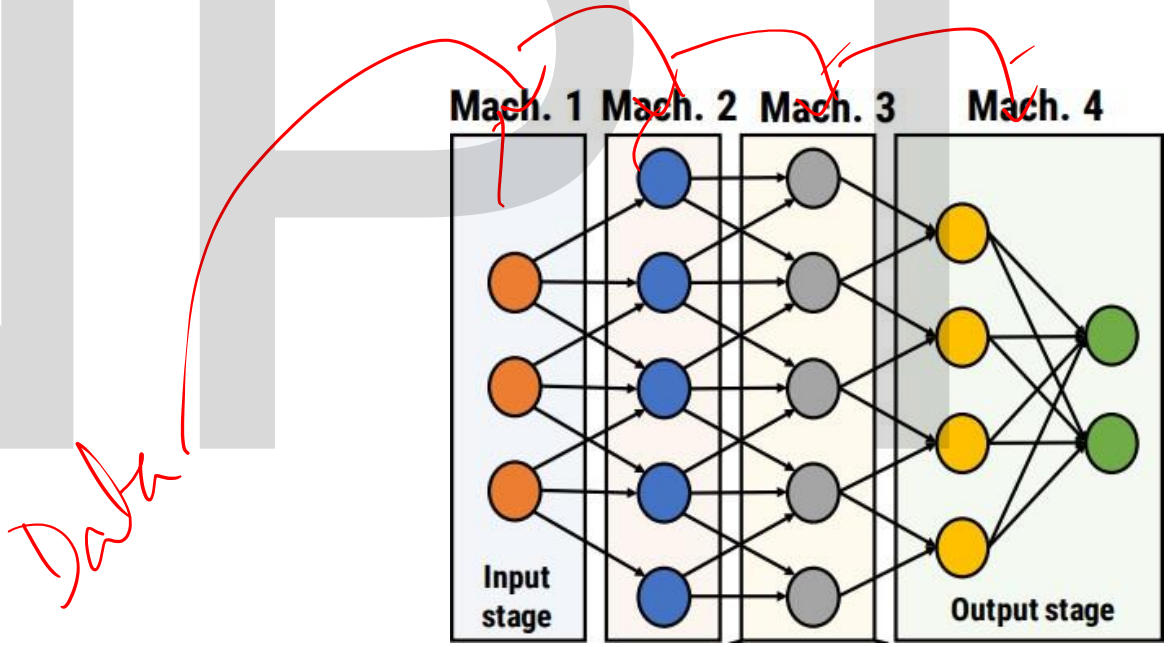
Distributed Training

- Data Parallelism
- **Model Parallelism**
- Pipeline Parallelism
- Tensor Parallelism

NPTEL

Model Parallelism

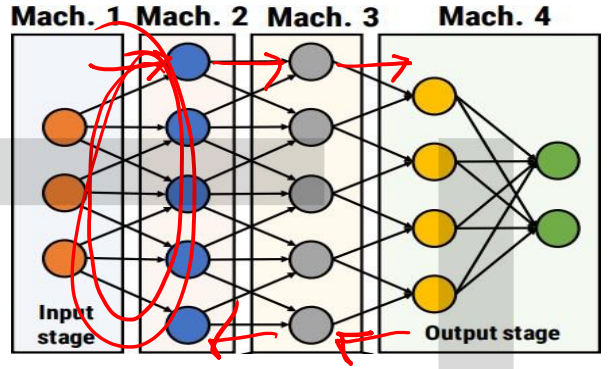
- Absolutely necessary when the model cannot be hosted in the memory of a single GPU
- Divide the model across machines and replicate the data

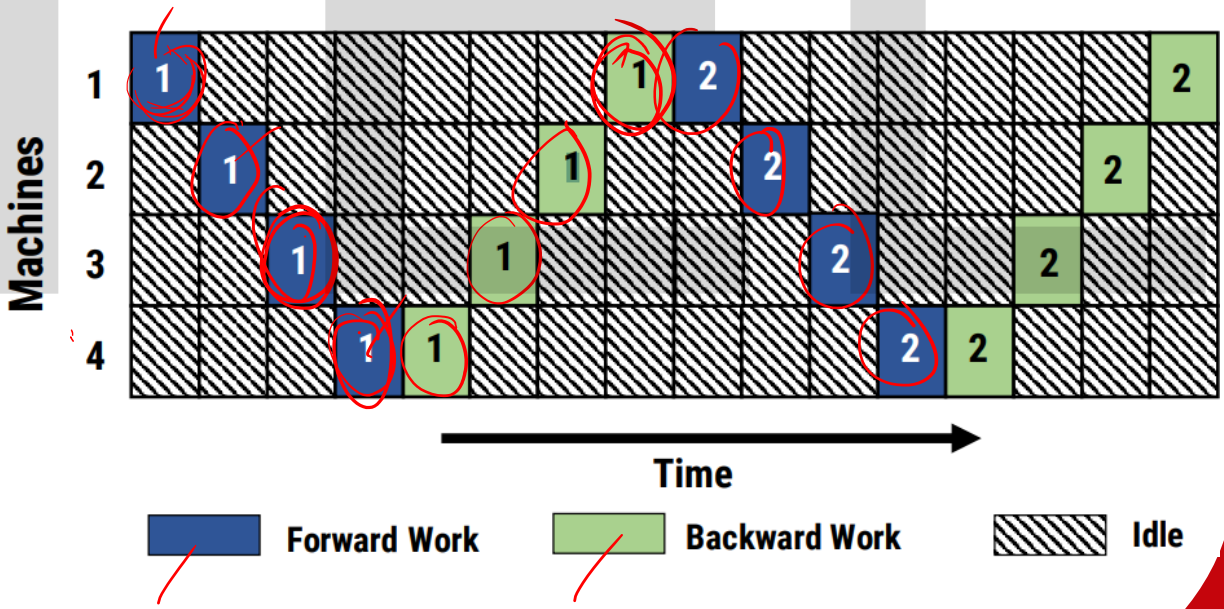


NPTEL

Model Parallelism

- **Advantages**
  - Increased total memory capacity -> model size
  - Point-to-point communication -> less expensive
  - Small minibatch: better statistical efficiency
- **Disadvantages**
  - Underutilization
  - Cannot mask communication
- **How to partition the model over the workers?**





NPTEL

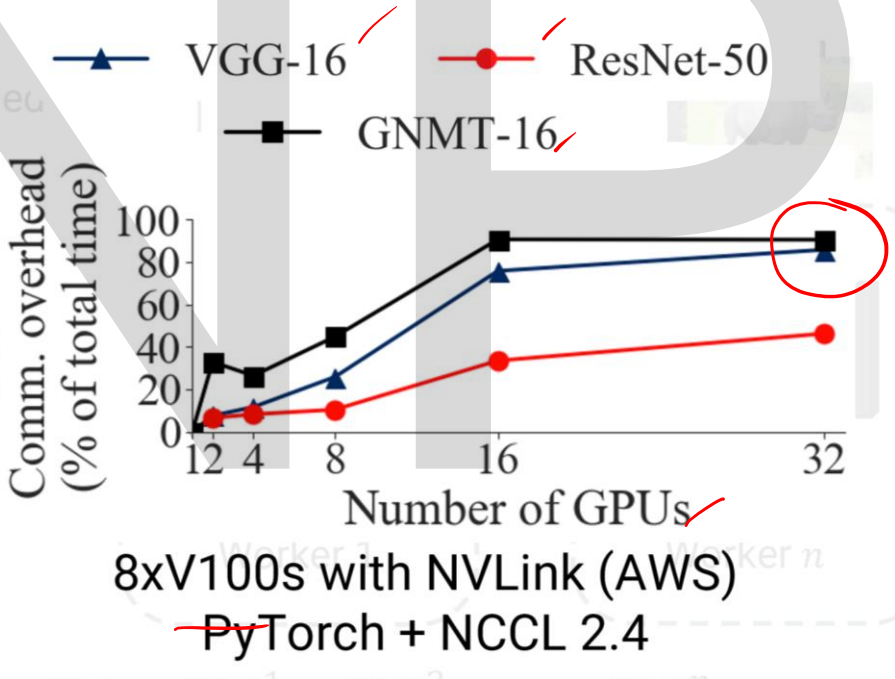
Distributed Training

- Data Parallelism
- Model Parallelism
- **Pipeline Parallelism**
- Tensor Parallelism

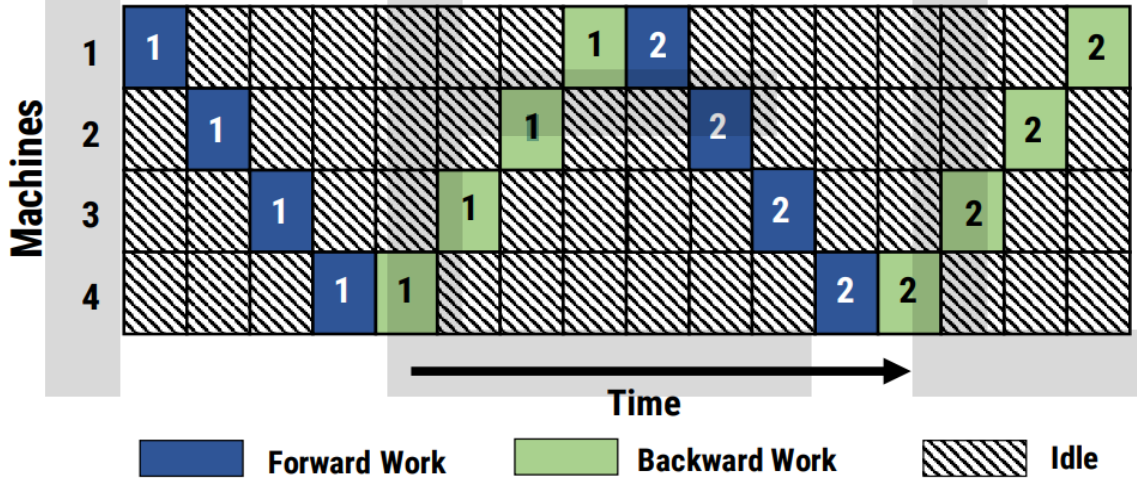
NPTEL

Pipeline Parallelism

DP : Communication overhead

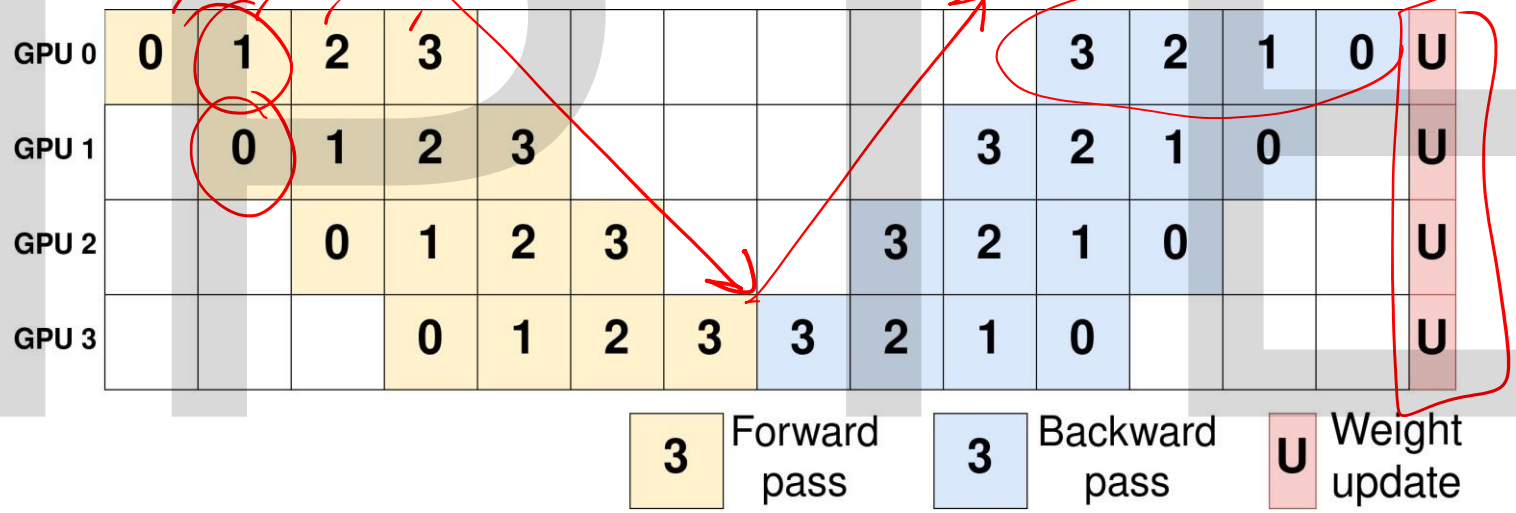


MP : Poor resource utilization – Low throughput



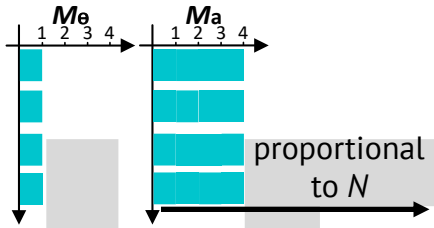
Pipeline Parallelism

- Similar to model parallelism, but use multiple batches in a pipelined fashion

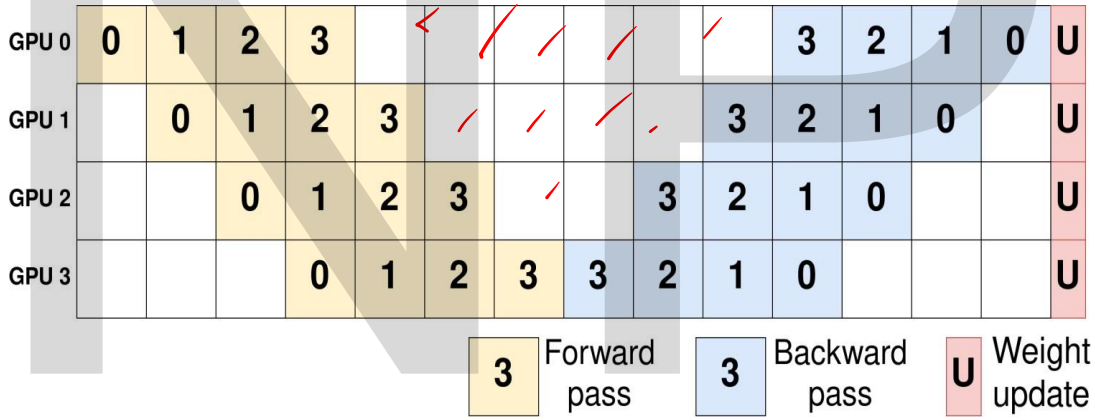


NPTEL

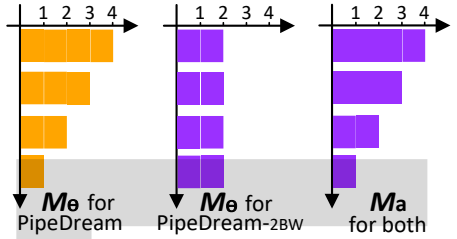
Variants of Pipeline Parallelism



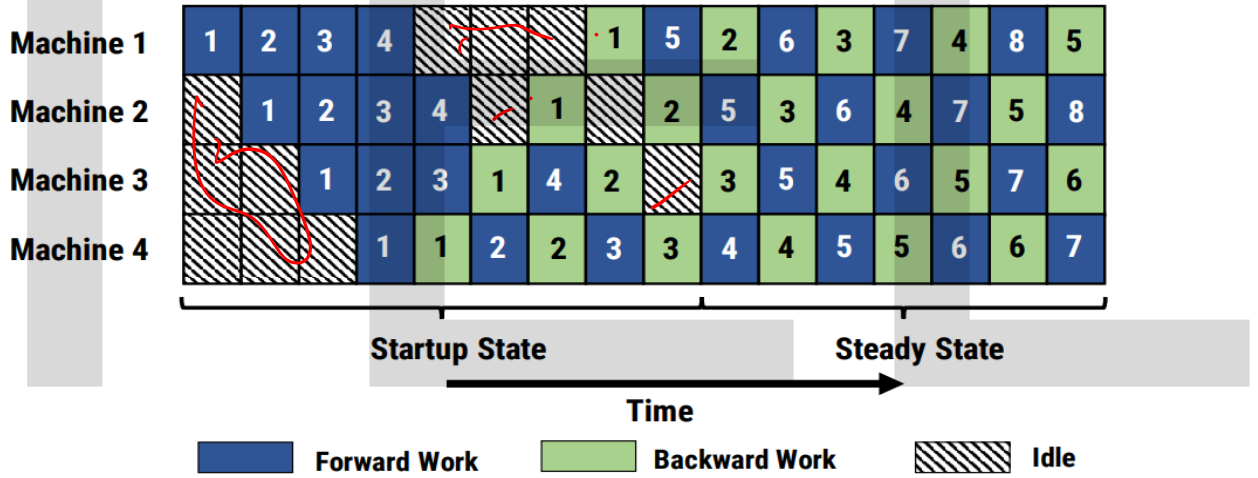
Intra-batch PP (Gpipe – NeurIPS'19)



- Microbatches help reduce bubbles compared to MP, but pipeline bubbles still exist!
- No staleness, close to batch SGD



Inter-batch PP (Pipedream – SOSP'19)



- Pipeline bubbles are minimized
- Staleness introduced

Challenges with Pipeline Parallelism

- **Work Partitioning**
  - Minimize load imbalance between the partitioned layers at each worker
  - Minimize communication across workers
- **Scheduling FW/BW pass**
  - Schedule an in-flight BP or new FP?
- **Staleness and convergence**

NPTEL

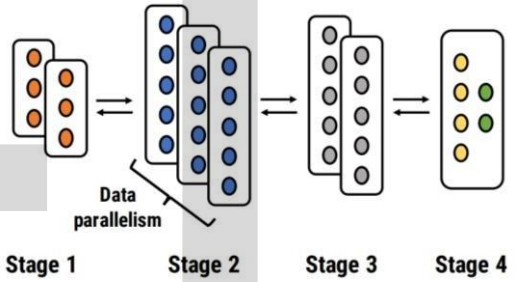
Work Partitioning

GPipe

- Manual
- Get per-layer profile and identify the right layer partitioning

Pipedream

- Automated
- Per-layer latency



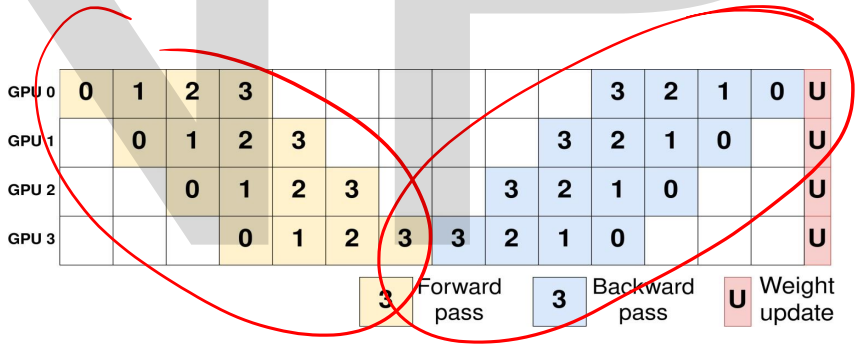
- Profiler + Optimizer
- Uses dynamic programming to find partition that minimizes overall train time

NPTEL

Scheduling

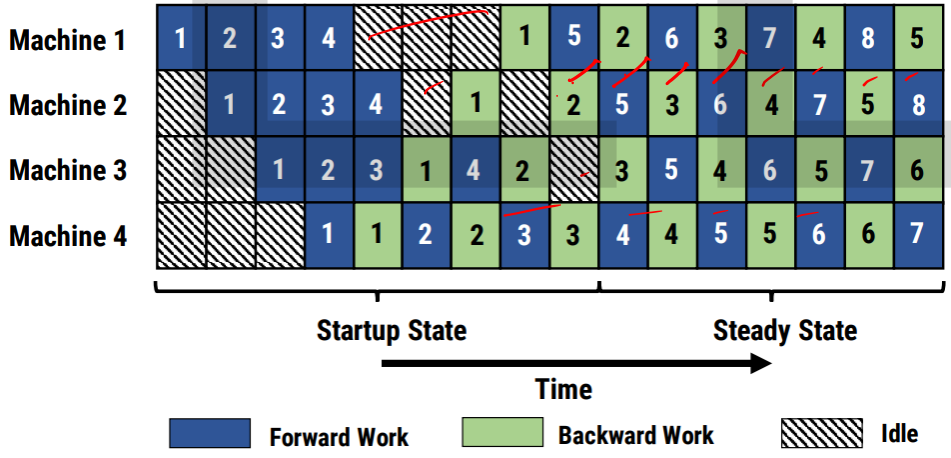
GPipe

- Forward for all microbatches
- Followed by backward for all microbatches



Pipedream

- Simple static 1F1B work scheduling:
- Alternate FW and BW passes
- Keeps pipeline full after warmup
- Static -> cheap



NPTEL

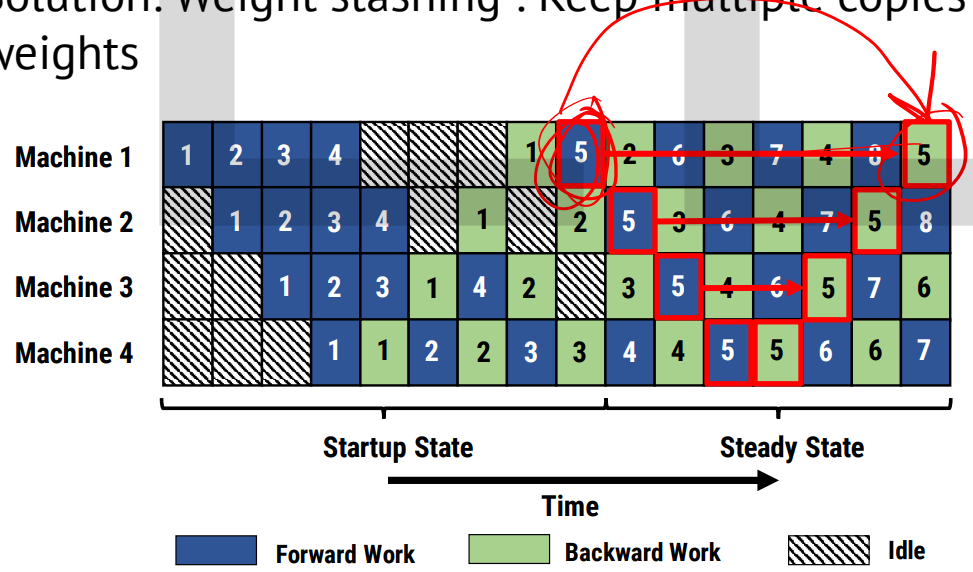
Convergence

GPipe

- Unaffected
- Similar to Data Parallelism with a batch
  B = microbatchsize * workers

Pipedream

- FW and BW pass for 1 minibatch performed with different weight versions
- Hinders model convergence
- Solution: Weight stashing : Keep multiple copies of weights



NPTEL

Word of Caution!

- **General advice: Training method that adversely affect generalization are not adopted, unless there is a 10x speed improvement**
- **Otherwise, there are so many parts moving that can go wrong in training large models, that most often practitioners stay away from async methods unless absolutely necessary**
  - E.g., training a very large recommendation system

NPTEL

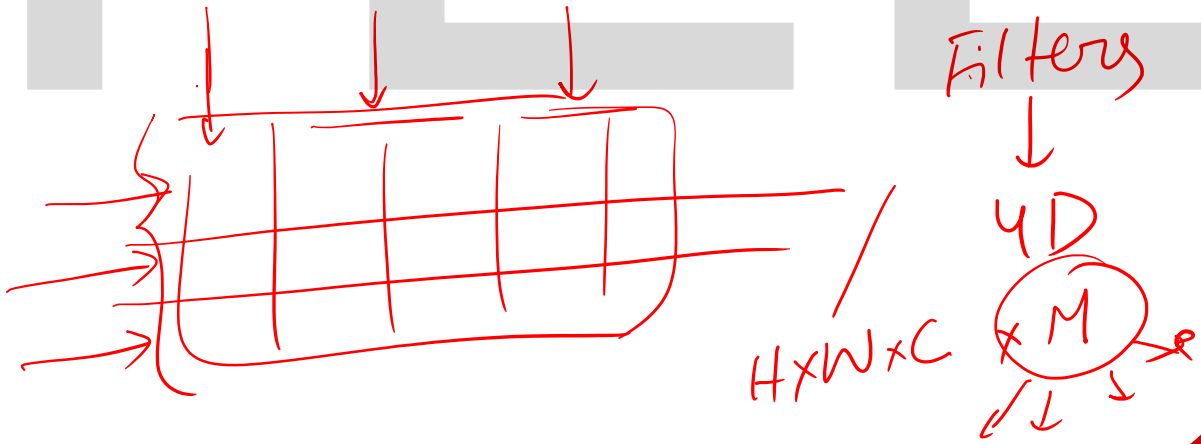
Distributed Training

- Data Parallelism
- Model Parallelism
- Pipeline Parallelism
- **Tensor Parallelism**

NPTEL

Tensor Parallelism

- **Model parallelism, Pipeline Parallelism split layers across workers (GPUs)**
- **What if a layer itself is so large that a GPU cannot handle it?**
- **Tensor parallelism**
  - Split a tensor into N chunks along a specific dimension and each device only holds 1/N of the whole tensor while not affecting the correctness of the computation graph
  - This requires additional communication to make sure that the result is correct



NPTEL



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)